Interface:

In Java, an **interface** is a reference type, similar to a class that can contain only constants, method signatures, default methods, static methods, and nested types. Interfaces cannot contain instance fields or constructors. The methods in interfaces are by default **abstract** (unless specified as default or static).

Here's a basic explanation and a simple example of an interface in Java:

Key Points:

1. **Abstract Methods:** By default, all methods in an interface are abstract (i.e., they do not have a body). A class that implements an interface must provide implementations for all of its methods.
2. **No Constructors:** Interfaces cannot have constructors because they cannot be instantiated directly.
3. **Multiple Inheritance:** A class can implement multiple interfaces, which is Java's way of supporting multiple inheritance.
4. **Default Methods:** From Java 8 onwards, interfaces can have default methods with a body, which allows a class to use the method without needing to implement it.

Example:

```
// Define an interface
interface Animal {
    // Abstract method (does not have a body)
    void sound();

    // Default method
    default void sleep() {
        System.out.println("This animal is sleeping.");
    }
}

// Implement the interface in a class
class Dog implements Animal {
    // Provide implementation for abstract method
    @Override
    public void sound() {
        System.out.println("Woof");
    }
}

// Another class implementing the interface
class Cat implements Animal {
    @Override
    public void sound() {
        System.out.println("Meow");
    }
}
```

```
public class Main {
    public static void main(String[] args) {
        // Create objects of Dog and Cat
        Dog dog = new Dog();
        Cat cat = new Cat();

        // Call methods
        dog.sound();  // Output: Woof
        dog.sleep();  // Output: This animal is sleeping.

        cat.sound();  // Output: Meow
        cat.sleep();  // Output: This animal is sleeping.
    }
}
```

Interface Definition in Java with Constants

In Java, an **interface** can define **constants**. Constants in interfaces are implicitly `public`, `static`, and `final`. This means that the constants are shared across all implementations of the interface and cannot be modified. Constants are typically used for defining fixed values that are relevant across various implementations.

Key Points:

1. **Constants** in interfaces are always `public`, `static`, and `final`.
2. They are accessible using the interface name (e.g., `InterfaceName.CONSTANT`).
3. Constants provide a way to avoid "magic numbers" in the code, making the program more readable and maintainable.

Syntax for Constants in Interface:

```
interface InterfaceName {
    // Constant declaration
    public static final type CONSTANT_NAME = value;
}
```

```
// Define an interface with constants
interface Vehicle {
    // Constants in the interface
    int MAX_SPEED = 120; // public static final by default
    String TYPE = "Automobile";

    // Abstract method
    void start();
    void stop();
}

// Implement the interface in a class
class Car implements Vehicle {
```

```
    // Implementing abstract methods
    public void start() {
        System.out.println("The car is starting.");
    }

    public void stop() {
        System.out.println("The car is stopping.");
    }

    // Additional method to access constant
    public void printVehicleInfo() {
        System.out.println("Vehicle Type: " + TYPE);
        System.out.println("Max Speed: " + MAX_SPEED + " km/h");
    }
}

public class Main {
    public static void main(String[] args) {
        // Create a Car object
        Car car = new Car();

        // Call methods
        car.start();          // Output: The car is starting.
        car.stop();           // Output: The car is stopping.
        car.printVehicleInfo();   // Output: Vehicle Type: Automobile, Max Speed: 120 km/h
    }
}
```